بخش هفتم: مثال‌های سیستم‌عاملی - لینوکس (Linux Scheduling)

(بر اساس فایل a.pdf7)

۱. مثال زمان‌بندی بلادرنگ در POSIX

استاندارد POSIX توابعی برای مدیریت سیاست زمان‌بندی نخ‌ها ارائه می‌دهد:

- **توابع کلیدی:**
  - pthread_attr_setschedpolicy: تنظیم سیاست (مثلاً به FIFO یا Round Robin).
  - pthread_attr_getschedpolicy: دریافت سیاست فعلی.

کد نمونه (مفهومی): در برنامه می‌توانیم با استفاده از این توابع، سیاست زمان‌بندی را روی SCHED_FIFO (اولین ورود-اولین سرویس بلادرنگ) تنظیم کنیم. اگر خطا ندهد، نخ‌های ایجاد شده با این سیاست اجرا می‌شوند که اولویت بالاتری نسبت به نخ‌های معمولی دارند.

۲. تکامل زمان‌بند لینوکس

سیستم‌عامل لینوکس در طول زمان الگوریتم‌های مختلفی را تجربه کرده است:

نسخه‌های اولیه (قبل از ۲.۵):
- از الگوریتم سنتی یونیکس استفاده می‌کرد.
- مشکل: روی سیستم‌های چندپردازنده (SMP) مقیاس‌پذیر نبود.

نسخه ۲.۵ (زمان‌بند O(1)):
- زمان انتخاب وظیفه بعدی **ثابت** بود و به تعداد پردازه‌ها ربطی نداشت.
- مشکل: برای کارهای تعاملی (Interactive) و دسکتاپ عملکرد روانی نداشت (زمان پاسخ ضعیف).

نسخه ۲.۶.۲۳ به بعد (زمان‌بند CFS):
- معرفی **Completely Fair Scheduler** (زمان‌بند کاملاً منصف).
- الگوریتم پیش‌فرض فعلی لینوکس.

۳. زمان‌بند کاملاً منصف (CFS)

ایده اصلی CFS این است که به جای اختصاص "تایم اسلایس" ثابت، به هر پردازه **سهمی (Proportion)** از پردازنده را بر اساس اولویتش می‌دهد.

مفاهیم کلیدی CFS:

مقدار Nice (ادب):

- اولویت در لینوکس با مقدار nice شناخته می‌شود.
- بازه: **۲۰- تا ۱۹+**
- **۲۰-**: بالاترین اولویت (بیشترین سهم CPU).
- **۱۹+**: پایین‌ترین اولویت (کمترین سهم CPU).
- پیش‌فرض: ۰.

تأخیر هدف (Target Latency):

- بازه زمانی که در آن تمام وظایف قابل‌اجرا باید حداقل یک بار اجرا شوند.
- این مقدار جایگزین برش‌های زمانی گسسته شده است.

زمان اجرای مجازی (vruntime): ⭐ *بسیار مهم*

- CFS زمان اجرای فیزیکی هر پردازه را اندازه می‌گیرد و آن را در متغیر vruntime ذخیره می‌کند.
- **نحوه محاسبه:** زمان واقعی اجرا ضرب‌در یک ضریب وزنی (بر اساس nice).
  - پردازه با اولویت بالا -> vruntime کندتر رشد می‌کند (دیرتر جریمه می‌شود).
  - پردازه با اولویت پایین -> vruntime سریع‌تر رشد می‌کند.
- **قانون انتخاب:** زمان‌بند همیشه پردازه‌ای را انتخاب می‌کند که **کوچک‌ترین vruntime** را دارد.

۴. پیاده‌سازی CFS (درخت قرمز-سیاه)

برای اینکه لینوکس بتواند سریعاً پردازه‌ای با کمترین vruntime را پیدا کند، از ساختمان داده **درخت قرمز-سیاه (Red-Black Tree)** استفاده می‌کند.

- این یک درخت جستجوی دودویی متوازن است.
- **کلید مرتب‌سازی:** مقدار vruntime.
- پردازه‌ها در درخت قرار می‌گیرند:
  - پردازه‌هایی که نیاز به CPU دارند (vruntime کم) -> سمت **چپ** درخت.
  - پردازه‌هایی که زیاد CPU مصرف کرده‌اند (vruntime زیاد) -> سمت **راست** درخت.
- **انتخاب بعدی:** چپ‌ترین گره (Leftmost Node) درخت که کمترین مقدار را دارد.

- **پیچیدگی زمانی:** O(log N). (البته لینوکس اشاره‌گر به چپ‌ترین گره را کش می‌کند تا دسترسی O(1) باشد).

۵. اولویت‌بندی کلی در لینوکس

لینوکس دو کلاس اصلی برای زمان‌بندی دارد که روی یک بازه عددی سراسری نگاشت می‌شوند:

زمان‌بندی بلادرنگ (Real-Time):

- شامل کلاس‌های SCHED_FIFO و SCHED_RR.
- بازه اولویت: **۰ تا ۹۹**.
- (عدد بزرگتر = اولویت بالاتر).

زمان‌بندی نرمال (Normal/CFS):

- شامل کلاس SCHED_OTHER (همان CFS).
- بازه اولویت: **۱۰۰ تا ۱۳۹**.
- این بازه دقیقاً با مقادیر nice (از ۲۰- تا ۱۹+) نگاشت می‌شود.
  - nice -20 -> اولویت ۱۰۰ (بالاترین در نرمال).
  - nice +19 -> اولویت ۱۳۹ (پایین‌ترین).

نکته مهم: پردازه‌های بلادرنگ (۹۹-۰) همیشه اولویت بالاتری نسبت به پردازه‌های نرمال (۱۳۹-۱۰۰) دارند و اگر وجود داشته باشند، حتماً اجرا می‌شوند.

۶. تعادل بار (Load Balancing) در لینوکس

زمان‌بند CFS طوری طراحی شده که از معماری‌های پیچیده سخت‌افزاری (مثل NUMA و چند هسته‌ای) آگاه باشد.

سلسله‌مراتب دامنه‌ها (Scheduling Domains): هسته‌ها در گروه‌هایی دسته‌بندی می‌شوند:

1. **سطح اول:** جفت نخ‌های سخت‌افزاری (SMT) در یک هسته.
2. **سطح دوم:** هسته‌هایی که کش 2L مشترک دارند.
3. **سطح سوم (NUMA Node):** پردازنده‌هایی که به حافظه محلی یکسان دسترسی دارند.

- **استراتژی تعادل بار:**
  - CFS سعی می‌کند بار را ابتدا در پایین‌ترین سطح متعادل کند.

- از جابجایی پردازه بین گره‌های NUMA (که هزینه دسترسی حافظه زیادی دارد) اجتناب می‌کند، مگر اینکه عدم تعادل خیلی شدید باشد.

پایان بخش a۷ (لینوکس) از فصل ۵